

Университет ИТМО

Факультет программной инженерии и компьютерной техники

Группа: Р3311

Студент: Горошников Т.И.

Преподаватель: Наумова Н.А.

Отчет по лабораторной работе №4

Задание

С помощью программного пакета Apache JMeter провести нагрузочное и стресс-тестирование веб-приложения в соответствии с вариантом задания.

В ходе нагрузочного тестирования необходимо протестировать 3 конфигурации аппаратного обеспечения и выбрать среди них наиболее дешёвую, удовлетворяющую требованиям по максимальному времени отклика приложения при заданной нагрузке (в соответствии с вариантом).

В ходе стресс-тестирования необходимо определить, при какой нагрузке выбранная на предыдущем шаге конфигурация перестаёт удовлетворять требованиями по максимальному времени отклика. Для этого необходимо построить график зависимости времени отклика приложения от нагрузки.

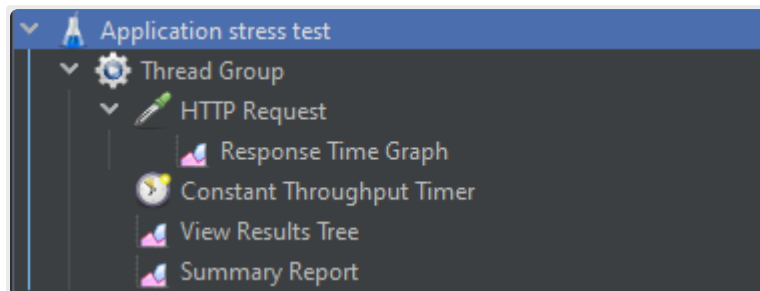
Параметры тестируемого веб-приложения:

- URL первой конфигурации (\$ 2300) — <http://stload.se.ifmo.ru:8080?token=518473098&user=-1355039501&config=1;>
- URL второй конфигурации (\$ 2900) — <http://stload.se.ifmo.ru:8080?token=518473098&user=-1355039501&config=2;>
- URL третьей конфигурации (\$ 3200) — <http://stload.se.ifmo.ru:8080?token=518473098&user=-1355039501&config=3;>
- Максимальное количество параллельных пользователей - 10;
- Средняя нагрузка, формируемая одним пользователем - 20 запр. в мин.;
- Максимально допустимое время обработки запроса - 530 мс.

Ход работы

Нагрузочное тестирование

Конфигурация нагрузочного тестирования



Разработанный тестовый план для нагрузочного тестирования

Thread Group

Name:

Comments:

Action to be taken after a Sampler error

☒ Continue ☐ Start Next Thread Loop ☐ Stop Thread ☐ Stop Test ☐ Stop Test Now

Thread Properties

Number of Threads (users):

Ramp-up period (seconds):

Loop Count: ☐ Infinite

☒ Same user on each iteration

☐ Delay Thread creation until needed

☐ Specify Thread lifetime

Duration (seconds):

Startup delay (seconds):

Конфигурация группы потоков для установки максимального числа одновременных пользователей

HTTP Request

Name:

Comments:

Basic Advanced

Web Server

Protocol [http]: Server Name or IP: Port Number:

HTTP Request

GET Path: Content encoding:

☐ Redirect Automatically ☒ Follow Redirects ☒ Use KeepAlive ☐ Use multipart/form-data ☐ Browser-compatible headers

Parameters Body Data Files Upload

Send Parameters With the Request:

Name:	Value	URL Encode?	Content-Type	Include Equals?
token	518473098	☐	text/plain	☒
user	-1355039501	☐	text/plain	☒
config	1	☐	text/plain	☒

Конфигурация HTTP запросов к приложению

В качестве имени сервера указывается localhost, а в качестве порта 8081, поскольку для подключения к приложению используется проброс портов на кафедральную сеть:

```
ssh -p2222 -L 8081:stload.se.ifmo.ru:8080 s408476@helios.se.ifmo.ru
```

Constant Throughput Timer

Name:

Comments:

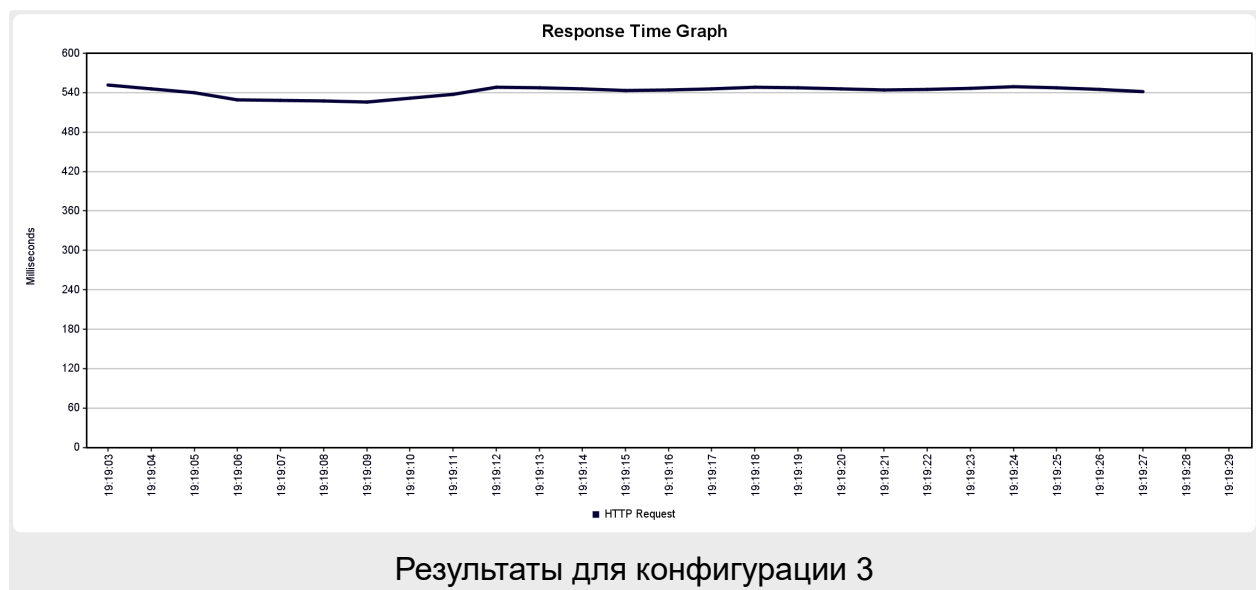
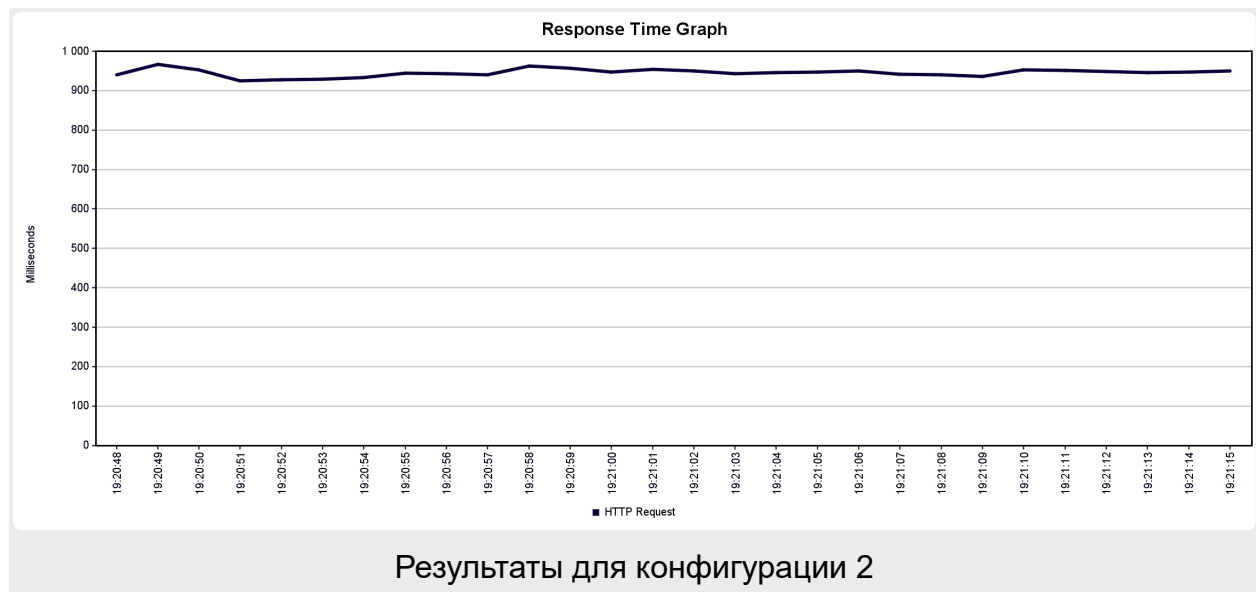
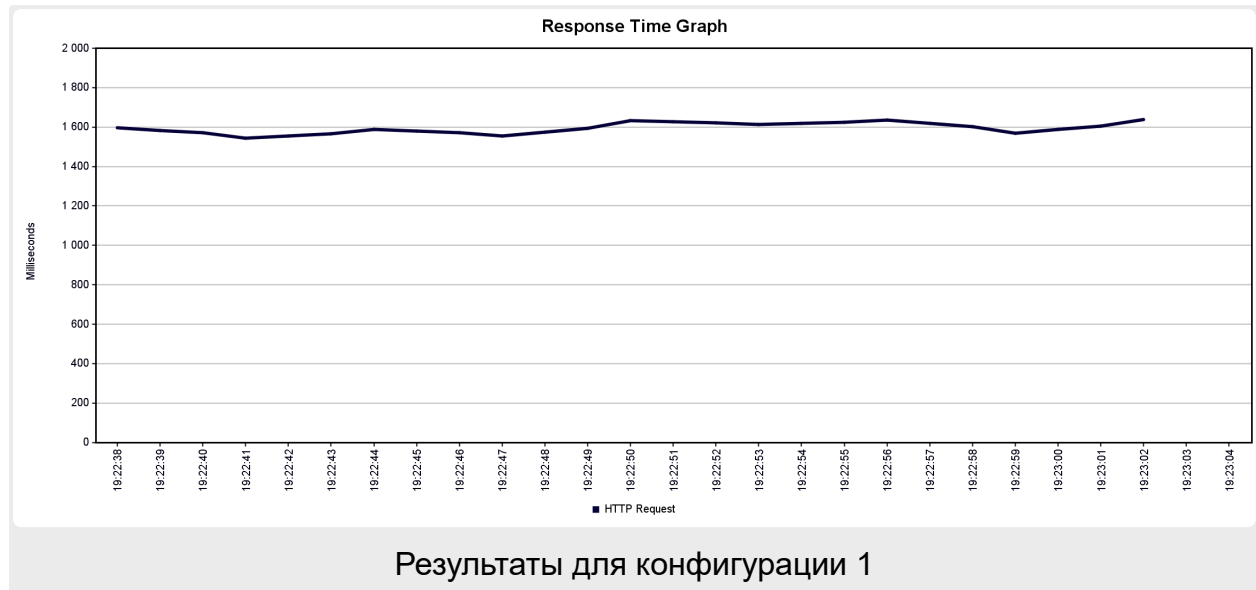
Delay before each affected sampler

Target throughput (in samples per minute):

Calculate Throughput based on:

Конфигурация таймера для установки среднего RpM

Результаты нагрузочного тестирования



Результаты нагрузочного тестирования показывают, что приложение лучше всего справляется с нагрузкой в третьей аппаратной конфигурации.

Summary Report

Name:

Summary Report

Comments:

Write results to file / Read from file

Filename

Browse...

Log/Display Only:

☐ Errors

☐ Successes

Configure

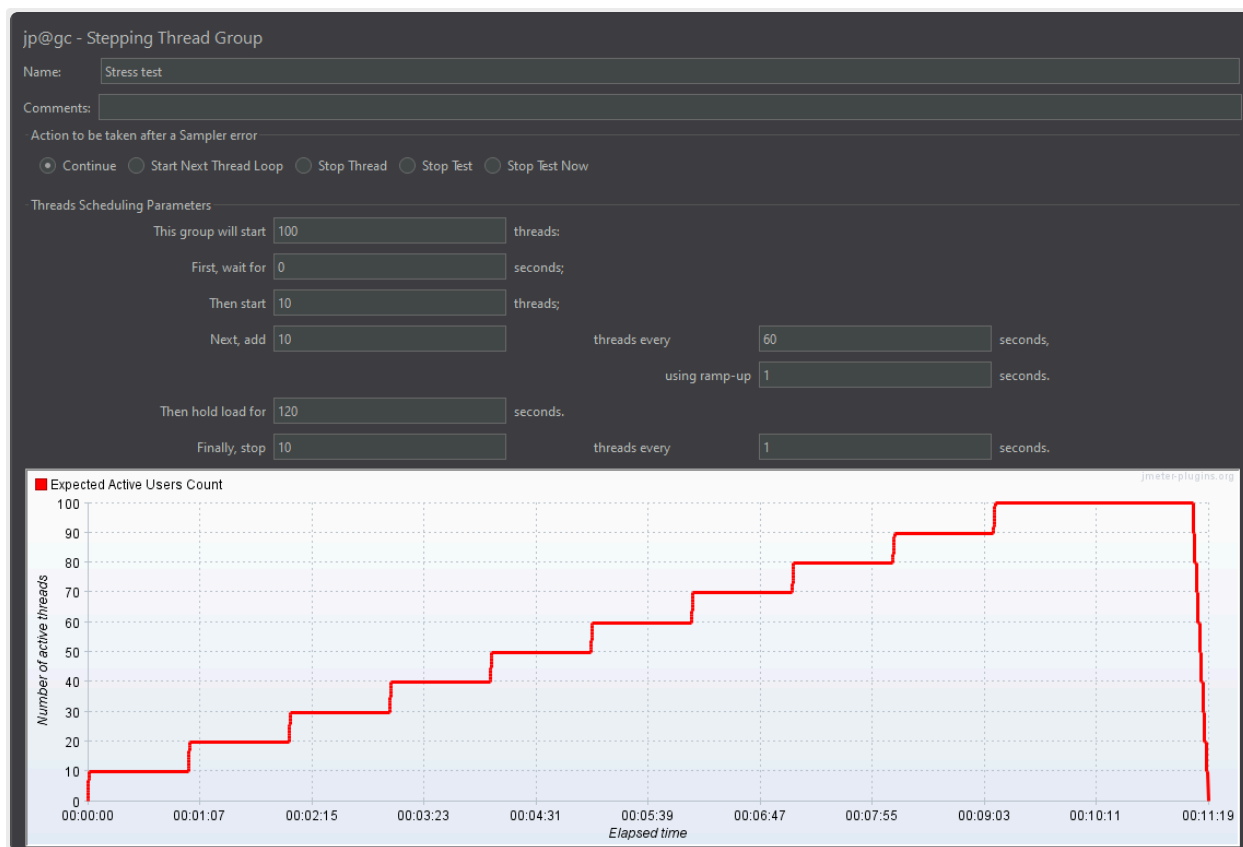
Label	# Samples	Average	Min	Max	Std. Dev.	Error %	Throughput	Received KB/sec	Sent KB/sec	Avg. Bytes
HTTP Request	100	545	512	629	17,16	0,00%	3,5/sec	0,79	0,55	231,0
TOTAL	100	545	512	629	17,16	0,00%	3,5/sec	0,79	0,55	231,0

Сводка результатов нагрузочного тестирования приложения в третьей конфигурации

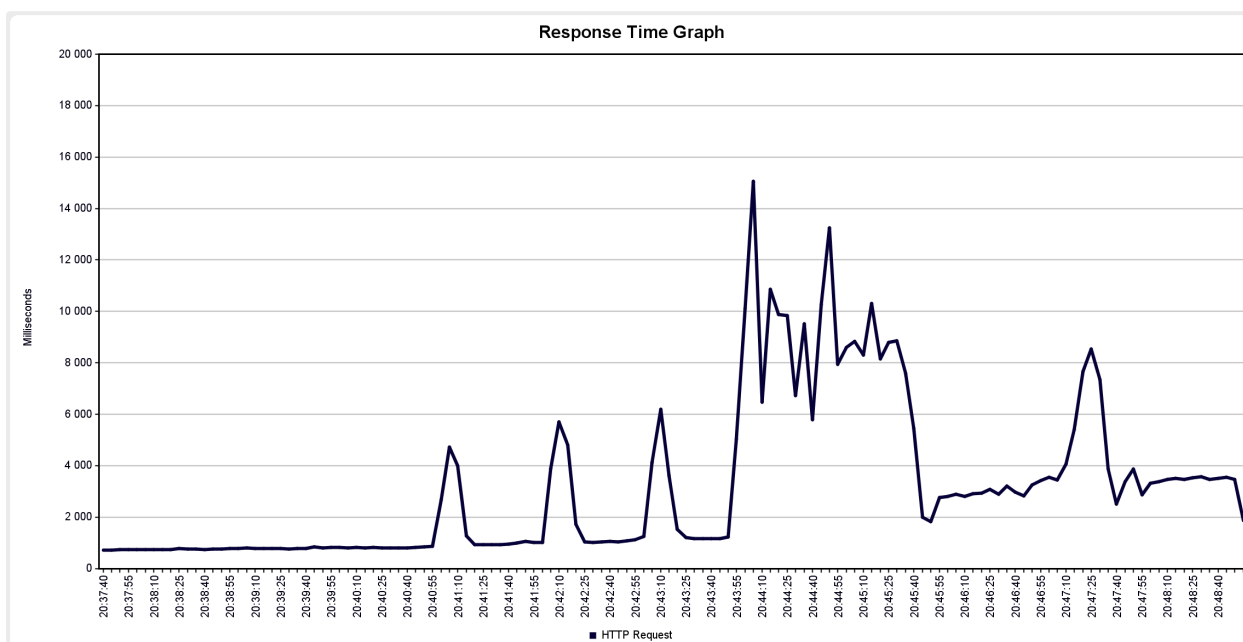
Время обработки запросов в данной конфигурации в среднем превышает заданное требованиями время на ~10-15мс, что незначительно (на мой взгляд).

Стресс-тестирование

Для стресс-тестирования я установил плагин Custom Thread Groups и использовал группу потоков Stepping Thread Group из него:



Конфигурация группы потоков для постепенного увеличения числа
одновременных пользователей



Результаты стресс-тестирования приложения

По графику мы можем заметить, что вначале приложение стабильно справляется с нагрузкой.

При увеличении числа пользователей до 50-60 приложение начинает давать сбои, что отражается резкими скачками на графике (т.е. резко возрастающим временем ожидания ответа). При дальнейшем увеличении числа пользователей приложение начинает работать нестабильно, что характеризуется изломанной областью на графике (20:43:55-20:45:40).

После того, как новые пользователи перестают подключаться (их число фиксируется на 100) работа приложения относительно стабилизируется, но время отклика все еще чересчур высокое (превышает заданное время отклика в 6-7 раз) и присутствует скачок на 20:47:25.

Из данного анализа можно сделать следующий вывод:

Число пользователей	Среднее время отклика	Примерное отклонение от заданного	Вывод
1-20	545 мс	15 мс	Оптимальное число одновременных пользователей для стабильной работы приложения
21-50	850 мс	320 мс	Допустимое число одновременных пользователей, однако время отклика уже заметно выше заданного
51-70	2000 мс	1470 мс	Возможно нестабильное поведение системы, недопустимое время отклика
71-100	5500 мс	4970 мс	Нестабильное поведение системы, абсолютно недопустимое время отклика

Вывод

В ходе данной лабораторной работы я научился использовать Apache JMeter для проведения нагрузочного и стресс-тестирования приложений. Я познакомился с различными инструментами для задания и мониторинга нагрузки, а также для визуализации результатов тестирования.